

Total Activation and the iCAPy Toolbox

Python Implementation

Version 1.0
April 3, 2026

Contents

1 License	1
2 Introduction	1
3 Example Data	2
4 Total Activation	2
4.1 Data Input and Preprocessing Options	3
4.2 TA-related Options	3
5 Thresholding	4
5.1 Thresholding-related Options	4
6 Clustering	4
6.1 Aggregating Data	4
6.2 Clustering	5
6.3 Consensus Clustering	5
6.4 Cluster Consensus	5
6.5 Clustering-related Options	5
7 Time Courses	6
7.1 Time Courses-related Options	6
7.2 Transient-informed Regression	6
7.3 Unconstrained Regression	7
References	8

1 License

This toolbox is shared under the GPLv3 License (<https://opensource.org/licenses/gpl-3.0>).

2 Introduction

This document gives a step by step guide on how to use the total activation (TA) and innovation-driven co-activation patterns (iCAPy) toolbox in Python. Theory and background of the algorithms can be found in [Karahanoglu et al., 2011, Karahanoglu et al., 2013, Karahanoglu and Van De Ville, 2015, Farouj et al., 2017, Zoller et al., 2018].

Main functionalities of the toolbox include:

- Total activation for regularized deconvolution of fMRI data
- Thresholding of innovation frames based on surrogate data
- Clustering of innovation frames to extract spatial brain activity patterns
- Spatio-temporal regression for time course recovery of networks

For each step, there exists a script `main_[stepname].py`, which first calls a script `Inputs_[stepname].py` and `Inputs_[stepname]_data.py` to specify the parameters, and then the function `Run_[stepname].py` to execute the selected step according to the specified parameters. All parameters are stored in a Python dictionary called `param`.

3 Example Data

The toolbox includes scripts and results for an example resting-state fMRI dataset. The example data was obtained from the OpenfMRI database (<https://openfmri.org/dataset/ds000030/>) [Poldrack et al., 2016] and pre-processed using Statistical Parametric Mapping (SPM12). Initial preprocessing steps included realignment, spatial smoothing with an isotropic Gaussian kernel of 6 mm full-width half-maximum, co-registration of structural scans to the functional mean and segmentation with the SPM12 Segmentation algorithm [Ashburner and Friston, 2005]. Further motion correction steps are included in the total activation pipeline.

Look at the example main and input scripts to see how to run the complete pipeline on the example fMRI data. Note that the results are not stable for this few data — the example only exists to demonstrate how input and output data is saved. Resulting iCAPs and time courses are very noisy and unstable because there is not enough input data for the clustering. For stable results, more subjects or longer runs would be necessary.

4 Total Activation

This sub-routine reads input fMRI data, conducts preprocessing steps for motion correction according to the options specified by the user, and saves the results for every subject in a subdirectory `TA_results/<title of project>` in every subject's folder. After execution of the TA pipeline, this folder will contain the following subfolders:

- `inputData`: contains preprocessed input data
- `TotalActivation`: contains results of total activation on real data
- `Surrogate`: contains results of total activation on surrogate data

Both the TotalActivation and Surrogate folders also contain a rolling checkpoint file (ta_checkpoint.h5) during the run. This file is saved after every N iterations (controlled by param['checkpoint_every'] in Inputs_TA.py) and deleted automatically on successful completion. If the run is interrupted, it resumes automatically from the last saved checkpoint on restart.

4.1 Data Input and Preprocessing Options

Have a look at the file **Inputs_TA_data.py** for the required inputs related to fMRI data reading and preprocessing. If you don't want to do any motion correction (possibly if you have done it already beforehand), set the fields **param['doDetrend']** and **param['doScrubbing']** to 0.

Key data parameters include:

- **param['PathData']** — path to the root data directory
- **param['Subjects']** — list of subject subfolder names
- **param['TR']** — repetition time in seconds
- **param['Folder_functional']** — subfolder containing fMRI NIfTI files
- **param['TA_func_prefix']** — filename prefix for functional NIfTI files. Can be a single string shared across all subjects, a list with one element, or a list with one entry per subject
- **param['Folder_GM']** — subfolder containing the probabilistic gray matter map
- **param['TA_gm_prefix']** — filename prefix for the GM map NIfTI file. If the file is in structural resolution, it will automatically be resampled to functional resolution using `scipy.ndimage.affine_transform` and saved with an 'f' prefix for future runs

4.2 TA-related Options

Have a look at the script **Inputs_TA.py** for the list and explanation of all parameters related to total activation.

Key TA parameters include:

- **param['HRF']** — hemodynamic response function type: 'bold', 'spmhrf', or 'mion'
- **param['Nit']** — number of outer forward-backward splitting iterations (default 5)
- **param['NitTemp']** — starting number of temporal regularization iterations (incremented by 100 each outer iteration)
- **param['LambdaSpat']** — spatial regularization weight (default 6)
- **param['use_gpu']** — set to 1 to use CuPy GPU acceleration for the spatial step (requires pip install cupy-cuda12x) (**HIGHLY RECOMMEND**)
- **param['gpu_chunk_size']** — number of time points to transfer to GPU per batch; controls transfer overhead only, not output values. Start at 100 for an 8GB GPU
- **param['runParallel']** — set to 1 to use multiprocessing for the temporal step (recommended)
- **param['checkpoint_every']** — save checkpoint every N iterations (1 = after every iteration, 0 = disabled)

5 Thresholding

This sub-routine applies a two-step thresholding procedure to determine which innovation frames (in TotalActivation/Innovation.nii) contain significant transients. After execution of the thresholding procedure, the following folder will be created in each subject's TA_results/<title of project> subfolder:

- **Thresholding:** contains results from thresholding. A subfolder is created for each set of thresholding options.

5.1 Thresholding-related Options

Have a look at the file **Inputs_Thresholding_data.py** for the required inputs related to reading of TA results files. In **Inputs_Thresholding.py** all options for thresholding are provided and explained.

6 Clustering

Attention!

For clustering, all subjects must be normalized to the same space. If you have run TA/thresholding in the subject-specific space, you need to normalize the thresholded innovation frames to MNI before continuing!

Tip: create a separate folder with the normalized files in TA_results/<title of project>_MNI and save the normalized data in it with the correct structure (i.e., in a subfolder called TotalActivation). Like that you can simply change param['title'] to read the normalized files.

6.1 Aggregating Data

The clustering routine first reads and concatenates TA and thresholding results of all subjects. Concatenated files will be saved in a folder iCAPs_results/<title of project>_<thresholding specifications>. The following clustering input files will be saved:

- **final_mask** (.pkl and .nii): the final mask of voxels with data in all subjects. By default this is the intersection of all subjects' grey matter masks. See clustering options for additional masking options (common_mask_file, extra_mask_file)
- **AI** (N_timepoints x N_voxels): concatenated activity-inducing signals of all subjects
- **AI_subject_labels** (N_timepoints x 1): subject assignment of each row in AI (0-based)

- **I_sig** (N_significant_innovations x N_voxels): concatenated significant innovations for all subjects. Positive and negative innovations are split into separate frames
- **subject_labels** (N_significant_innovations x 1): subject assignment for each row of I_sig (0-based)
- **time_labels** (N_significant_innovations x 1): time point assignment for each row of I_sig

6.2 Clustering

After aggregating data from all subjects, k-means clustering is applied to significant innovation frames I_sig. Clustering results will be stored in a subfolder named according to the specific clustering options. The following outputs will be created:

- **iCAPs** (.pkl and .nii): cluster centers of all resulting clusters — the spatial iCAPs maps
- **iCAPs_z** (.pkl and .nii): z-scored cluster centers
- **dist_to_centroid** (N_significant_innovations x N_clusters): distances from every clustered innovation frame to each cluster center
- **IDX** (N_significant_innovations x 1): for each frame the index of the cluster it is assigned to (0-based)
- **param** (.pkl): dictionary containing all input parameters that led to these results

6.3 Consensus Clustering

If requested according to the options, consensus clustering [Monti et al., 2003] is applied to evaluate cluster stability and find the best number of clusters to consider. Consensus matrices for each requested cluster number will be saved in a consensus clustering subfolder, as well as CDF and AUC values. These files can be used according to [Monti et al., 2003] to decide on the best number of clusters.

6.4 Cluster Consensus

If clustering and consensus clustering have been done, the consensus of every cluster can be computed as the average consensus index between all pairs of frames belonging to the same cluster. Values range from 1 (all frames are always clustered together across resamples) to 0 (frames are never clustered together). Higher values indicate higher stability of the cluster.

6.5 Clustering-related Options

Have a look at the file **Inputs_Clustering_data.py** for the required inputs related to reading of TA and Thresholding results files. In **Inputs_Clustering.py** all options for clustering are provided and explained.

7 Time Courses

As the iCAPs' spatial maps are retrieved from significant innovations, time courses have to be extracted by back-projection to the activity-inducing signals. One can choose to apply transient-informed or unconstrained regression for time course retrieval.

7.1 Time Courses-related Options

Have a look at the file **Inputs_TimeCourses_data.py** for the required inputs related to reading of clustering results files. In **Inputs_TimeCourses.py** all options for time course retrieval are provided and explained.

7.2 Transient-informed Regression

If time courses are extracted by transient-informed regression [Zoller et al., 2018], a subfolder will be created in the corresponding clustering folder. It will contain the following files:

- Results for all requested soft assignment factors (see options):
 - **TC**: list of arrays with time courses of every subject
 - **TC_stats**: statistics of data fit (BIC, AIC, ...)
 - **tempChar**: list of dicts with temporal characteristics (duration, co-activations, ...) for every subject's time courses
- Files related to selection of best soft assignment factor:
 - **BICsum.svg**: sum of BIC across all subjects, knee point marked in red
 - **BICdist.svg**: distribution of BIC across all subjects, knee point marked in red
 - **AICsum.svg**: sum of AIC across all subjects, knee point marked in red
 - **AICdist.svg**: distribution of AIC across all subjects, knee point marked in red

Results for the best soft assignment factor according to BIC will be saved in the main iCAPs directory and will contain: TC (time courses), TC_stats (statistics: BIC/AIC etc.) and tempChar (temporal characteristics).

Computed temporal characteristics are:

- **nTP_sub**: total number of time points per subject
- **innov_counts_total / innov_counts_perc**: total number / percentage of significant innovation frames
- **TC_norm_thres**: z-scored and thresholded time courses
- **TC_active**: binarized time courses (+1/0/-1 for positive/none/negative activation)
- **coactive_iCAPs_total**: number of co-active iCAPs per time point
- **occurrences_(pos/neg)**: number of activation blocks (positive/negative activations)
- **duration_total_counts_(pos/neg)**: total number of active time points
- **duration_avg_counts_(pos/neg)**: average duration per occurrence

- **coupling_counts**: number of frames with co-activation for every pairwise combination of iCAPs
- **coupling_jacc**: coupling Jaccard index (overlap divided by activation union)
- **coupling_sameSign/diffSign_counts**: positive/negative couplings
- **coupling_sameSign/diffSign_jacc**: positive/negative coupling Jaccard index

For the computation of duration and couplings, only activation blocks of 2 time points or more have been considered.

7.3 Unconstrained Regression

We do not recommend this option, but provide the possibility to compare with [Karahanoglu and Van De Ville, 2015, Bolton et al., 2018]. If time courses are extracted by unconstrained regression, a subfolder TCs_unconstrained will be created in the corresponding clustering folder. It will contain the following files:

- **TC_unc**: list of arrays with time courses of every subject
- **TC_unc_stats**: statistics of data fit (BIC, AIC, ...)
- **tempChar_unc**: list of dicts with temporal characteristics for every subject's time courses

References

- [Ashburner and Friston, 2005] Ashburner, J. and Friston, K. J. (2005). Unified segmentation. *Neuroimage*, 26(3):839-851.
- [Bolton et al., 2018] Bolton, T. A. W., Tarun, A., Sterpenich, V., Schwartz, S., and Van De Ville, D. (2018). Interactions Between Large-Scale Functional Brain Networks are Captured by Sparse Coupled HMMs. *IEEE Trans. Med. Imaging*, 37(1):230-240.
- [Farouj et al., 2017] Farouj, Y., Karahanoglu, F. I., and Van De Ville, D. (2017). Regularized Spatiotemporal Deconvolution of fMRI Data Using Gray-Matter Constrained Total Variation. *Proc. 14th IEEE Int. Symp. Biomed. Imaging From Nano to Macro*, pages 472-475.
- [Karahanoglu et al., 2011] Karahanoglu, F. I., Bayram, I., and Van De Ville, D. (2011). A Signal Processing Approach to Generalized 1-D Total Variation. *IEEE Trans. Signal Process.*, 59(11):5265-5274.
- [Karahanoglu et al., 2013] Karahanoglu, F. I., Caballero-Gaudes, C., Lazeyras, F., and Van De Ville, D. (2013). Total activation: FMRI deconvolution through spatio-temporal regularization. *Neuroimage*, 73:121-134.
- [Karahanoglu and Van De Ville, 2015] Karahanoglu, F. I. and Van De Ville, D. (2015). Transient brain activity disentangles fMRI resting-state dynamics in terms of spatially and temporally overlapping networks. *Nat. Commun.*, 6:7751.
- [Monti et al., 2003] Monti, S., Tamayo, P., Mesirov, J., and Golub, T. (2003). Consensus Clustering: A Resampling-Based Method for Class Discovery and Visualization of Gene Expression Microarray Data. *Mach. Learn.*, 52(1):91-118.
- [Poldrack et al., 2016] Poldrack, R. A., Congdon, E., Triplett, W., Gorgolewski, K. J., Karlsgodt, K. H., Mumford, J. A., Sabb, F. W., Freimer, N. B., London, E. D., Cannon, T. D., and Bilder, R. M. (2016). A phenome-wide examination of neural and cognitive function. *Sci. Data*, 3:1-12.
- [Zoller et al., 2018] Zoller, D. M., Bolton, T. A. W., Karahanoglu, F. I., Eliez, S., Schaer, M., and Van De Ville, D. V. D. (2018). Robust recovery of temporal overlap between network activity using transient-informed spatio-temporal regression. Under Review.